AWS CERTIFIED MACHINE LEARNING ENGINEER — ASSOCIATE (MLA-C01)

Machine Learning on AWS
The Core 90%

Domains 1–4 · Data Preparation · Model Development · Deployment · Monitoring & Security

A self-study deck: read the slide, then read the Study Notes underneath it

Companion to the GenAI deck (the other ~10%). Built task-by-task against the official Exam Guide v1.0.

READ THIS FIRST

How this exam thinks — the four habits

1 The constraint is the last sentence

"...with the LEAST operational overhead" · "...MOST cost-effective" · "...in real time" · "...minimize downtime".
Read it first. It decides among the 2–3 options that all 'work'.

2 Managed beats hand-built

If a purpose-built AWS feature exists (Feature Store, Model Monitor, AMT), assembling it yourself from Lambda and glue code is the WRONG answer.

3 Numbers eliminate options

Payload sizes, time limits, latency targets, label counts. When the stem contains numbers, the answer is usually arithmetic elimination, not opinion.

4 Know what is OUT of scope

The guide excludes some famous services. If an option is out-of-scope for MLA-C01, it is a distractor:
GuardDuty · Inspector · Cognito · WAF · IoT Greengrass · Route 53 · Elastic Beanstalk

! Habit 4 is free marks: an out-of-scope service in the options is AWS telling you which answer to eliminate.

THE MENTAL MODEL

One map for the whole exam: the ML lifecycle

Every question lives at one station of this line. Locate the station first; the service candidates shrink instantly.

D1 · 28%

PREPARE

Ingest · store · transform
features · labels · integrity

→

D2 · 26%

BUILD

Choose algorithm · train
tune · evaluate · register

→

D3 · 22%

SHIP

Endpoints · containers · IaC
pipelines · CI/CD · strategies

→

D4 · 24%

OPERATE

Monitor drift · troubleshoot
costs · IAM · VPC · encryption

The loop that makes it MLOps:

OPERATE feeds back into PREPARE: Model Monitor detects drift → EventBridge fires → a SageMaker Pipeline re-runs PREPARE → BUILD → SHIP automatically. Half of Domains 3–4 is just wiring this loop. Keep the picture in your head.

THE SKILL THE EXAM ACTUALLY TESTS

Anatomy of a real MLA scenario — dissect, don't read

A retail company ingests 2 TB of clickstream data daily into Amazon S3. A data science team retrains a demand model weekly on a GPU instance; training takes 9 hours, and the first 2 hours are spent copying the dataset to the instance. **The team must reduce total training time. The solution must require minimal changes to the training code and must not increase storage costs. Which solution meets these requirements?**

Grey = NOISE

Company type, "2 TB", "weekly" — scene-setting. Locates the station (D1, data loading) and nothing else.

Colored = 3 CONSTRAINTS

1- cut time (the copy is the bottleneck)
2- minimal CODE changes
3- no extra STORAGE cost
Every option must clear ALL THREE.

Eliminate by constraint

Pipe mode fails 2 (stream-consuming rewrite)
Copy to FSx Lustre fails 3 (second copy)
Bigger instance fails 1 (copy time stays)
FastFile mode clears all three ✔

! The whole exam: locate the station → extract the 2–3 constraints → kill one option per constraint. Never "pick the best" — ELIMINATE.

1

Data Preparation for ML

28% of the exam — ingest, store, transform, label, protect

Where training data lives: S3 · EBS · EFS · FSx

Amazon S3 Object store — THE default data lake	Cheap, 11 nines, unlimited. Every SageMaker job reads/writes it. Boost long-distance uploads with S3 Transfer Acceleration.
Amazon EBS Block volume attached to ONE instance	The disk under a notebook/training instance. Need guaranteed IOPS? Provisioned IOPS volumes.
Amazon EFS Shared POSIX file system (NFS)	Many instances mount the SAME files — shared notebooks, shared datasets across training nodes.
FSx for Lustre High-throughput parallel FS	Feeds GPU clusters at speed; can LINK to an S3 bucket and lazy-load it as files.
FSx for NetApp ONTAP Managed NetApp	Named in the guide: enterprises with existing NetApp/ONTAP workflows.

! Default answer: S3. Deviate only on a stated need — shared POSIX (EFS), extreme GPU throughput (FSx Lustre), existing NetApp (ONTAP).

S3 for ML: classes, lifecycle, movement

Storage classes & lifecycle

Standard → hot training data.
Standard-IA → older datasets you keep but rarely touch.
Glacier tiers → archived raw data, compliance holds.

Lifecycle RULES move objects automatically by age — the exam's answer to "reduce storage cost of old training data."

Getting data INTO S3

AWS DataSync → scheduled/online transfer from on-prem NFS/SMB into S3/EFS/FSx.
Storage Gateway → hybrid: on-prem apps keep a local interface, data lands in S3.

Both are in-scope; their trigger is the phrase "from our on-premises file server."

Also useful:

Versioning (protect datasets from overwrite — reproducibility), server-side encryption with KMS (Domain 4 will insist on it), and partitioned layouts (s3://bucket/year=2026/month=07/...) so Athena and Glue scan less and cost less.

! "Cut the cost of storing old training data" → lifecycle rules to IA/Glacier. "Move data from on-prem NAS" → DataSync.

How training jobs READ data: File vs Pipe vs FastFile

File mode

Copy the WHOLE dataset from S3 to the instance disk, then start

Simple, default. Startup is slow and disk must fit the data.

Pipe mode

STREAM records from S3 into the algorithm as it trains

No download wait, no disk limit. Classic pairing: RecordIO-protobuf.

FastFile mode

Looks like files, streams like Pipe — on demand

Modern default for large data: file-API convenience + streaming start.

Beyond S3: mount Amazon EFS or FSx for Lustre directly as the training data channel when you need shared or ultra-fast file access — the guide names this configuration explicitly (Task 1.3).

Data formats: pick by ACCESS PATTERN

COLUMNAR — Parquet, ORC

Stores column-by-column, compressed, with schema.

Analytics reads FEW columns of MANY rows → scan less, pay less (Athena bills per byte scanned!), run faster.

Default for the lake and for feature tables.

ROW — CSV, JSON, Avro, RecordIO

CSV — universal exchange; no schema, no types, bulky.
JSON — nested/semi-structured; verbose.
Avro — row format WITH schema + evolution → streaming pipelines (Kafka/Kinesis).
RecordIO-protobuf — SageMaker built-ins' packed format; pairs with Pipe mode.

! "Reduce Athena query cost / speed up analytics on S3" → convert CSV/JSON to PARQUET (or ORC), partitioned.

The guide's phrase is "validated and non-validated formats": Parquet/ORC/Avro carry an enforced schema (validated); CSV/JSON carry none — anything can hide in them, which is why data-quality checks (later slide) exist.

Ingestion: batch or streaming?

BATCH — data at rest, on a schedule

Nightly loads, historical dumps, warehouse extracts.

Tools:
- AWS Glue jobs (serverless Spark ETL)
- Amazon EMR (managed big-data clusters)
- AWS DataSync (from on-prem shares)
- AWS Batch for arbitrary containerized jobs

Lands in S3 → catalogued → transformed.

STREAMING — data in motion, now

Clickstreams, IoT, transactions, logs.

Tools:
- Amazon Kinesis Data Streams / Firehose
- Amazon MSK (managed Apache Kafka)
- Managed Service for Apache Flink (stateful processing)
- Lambda for light per-record transforms

Next slide: choosing within this family.

! Words like "real-time", "as events occur", "clickstream" → streaming family. "Nightly / weekly / historical" → batch family.

The Kinesis decision: Streams vs Firehose

S Kinesis Data Streams

A durable, ordered LOG of records (shards).

YOU write consumers (Lambda, Flink, custom apps); multiple consumers replay the same stream; retention up to days.

Choose when: custom real-time processing, multiple readers, replay, ordering per key.

F Amazon Data Firehose

A DELIVERY PIPE: ingest → optional transform (Lambda) / format-convert to Parquet → land in S3, Redshift, OpenSearch.

Fully managed, near-real-time buffering, zero consumer code.

Choose when: "just get the stream into S3/warehouse".

Family footnotes: Kinesis Video Streams for camera/AV feeds (in scope). MSK = managed Kafka — pick it on the words "existing Kafka" or "Kafka APIs", otherwise Kinesis.

! "Deliver the clickstream to S3 as Parquet with least code" → FIREHOSE (it even converts the format for you).

Transforming the stream: Flink vs Lambda

Managed Service for Apache Flink

STATEFUL stream processing: windows, aggregations, joins across events, exactly-once semantics.

"Average sensor reading per device over 5-minute windows" · "detect a pattern across events" — anything that must REMEMBER previous records.

AWS Lambda

STATELESS per-record (or per-batch) transforms: parse, filter, enrich, mask a field, reshape JSON.

Also the inline transform hook inside Firehose.

Each invocation sees only its records — no memory across events.

! Needs memory across events (windows, sessions, joins) → FLINK. Touches each record independently → LAMBDA.

Spark Structured Streaming on EMR is the third option — pick it when the team already runs Spark and wants one engine for batch AND stream.

AWS Glue: the serverless ETL backbone

Glue ETL Jobs	Serverless Spark (or Python shell) jobs — transform, join, convert to Parquet, write back. Pay per DPU-hour, zero clusters.
Glue Crawlers	Scan S3/JDBC sources, INFER schema and partitions, and register tables automatically.
Glue Data Catalog	The central metastore those tables live in — shared by Athena, EMR, Redshift Spectrum, Lake Formation.
Glue Studio	Visual job authoring for the same ETL.

Glue vs EMR:

Glue = serverless, per-job, "least management" ETL. EMR = you want the CLUSTER: specific frameworks (Hive/Presto/HBase), long-running big-data platforms, deep Spark tuning, or Spot-heavy cost engineering on massive jobs.

D1 · TASK 1.2 · WHICH TOOL

The transform-tool decision: DataBrew vs Data Wrangler vs Glue vs EMR

"Business analyst / no code" · visual profiling & cleaning · 250+ built-in transforms	**AWS Glue DataBrew**
"Data scientist inside SageMaker Studio" · visual prep FOR ML · export to pipeline/Feature Store	**SageMaker Data Wrangler**
"Engineers, code-based serverless ETL at scale" · scheduled production pipelines	**AWS Glue jobs**
"Cluster control, Hive/Presto, petabyte Spark, Spot fleets"	**Amazon EMR**
"Light per-record cleanup on a stream"	**AWS Lambda**

! Same job, four tools — the exam differentiates by WHO does it and HOW MUCH code/scale the stem implies.

The classic trap pair:

DataBrew and Data Wrangler both say "visual data preparation". Differentiate by HOME: DataBrew lives in Glue for general analytics users; Data Wrangler lives in SageMaker Studio and exports straight into ML pipelines and Feature Store.

D1 · TASK 1.1–1.3

Query & govern the lake: Athena · Redshift · Lake Formation

A Amazon Athena

Serverless SQL directly ON S3 via the Glue Catalog. Pay per TB scanned.

Trigger: "ad-hoc SQL on data in S3, no infrastructure."

Cost lever: Parquet + partitions.

R Amazon Redshift

The WAREHOUSE: persistent cluster/serverless for heavy BI, complex joins, dashboards at scale.

Spectrum reaches into S3 too.

Trigger: "enterprise BI/warehouse workloads."

L AWS Lake Formation

GOVERNANCE over the lake: central grants, database/table/column-level (even row-level) permissions on Catalog data.

Trigger: "fine-grained access control across teams."

! "Different teams may see only certain COLUMNS of the lake" → Lake Formation permissions — not a pile of bucket policies.

SageMaker Data Wrangler: prep that flows into ML

1 · Import	S3, Athena, Redshift, Snowflake, Feature Store…
2 · Profile	Data Quality & Insights report: types, missing %, correlations, target leakage hints, quick-model score
3 · Transform	300+ built-ins: impute, encode, scale, balance (SMOTE), text featurize — or custom PySpark/Pandas
4 · Export	→ Processing job · → SageMaker Pipelines step · → Feature Store · → Python code

! Its superpower is the EXPORT: the visual recipe becomes a reusable pipeline step — prototype once, productionize unchanged.

The profile report even flags LIKELY TARGET LEAKAGE and class imbalance before you train — remember this when those topics appear two slides ahead.

SageMaker Feature Store: one definition, two stores

OFFLINE store

Full history in S3 (queryable via Athena).

For building TRAINING sets — including time-travel: "features as they were on the day of each label."

ONLINE store

Low-latency key-value lookup of the LATEST feature values.

For INFERENCE: the endpoint fetches the customer's current features in milliseconds.

Why it exists — the two diseases it cures:

1) TRAINING–SERVING SKEW: train-time and serve-time features computed by different code drift apart. One definition, written once, read by both.
2) DUPLICATED WORK: teams re-deriving "customer_30d_spend" five ways. Central, discoverable, versioned feature groups.

! "Reuse features across teams" + "consistent between training and real-time inference" → Feature Store, online + offline.

Labeling: SageMaker Ground Truth

Workforces	Amazon Mechanical Turk (public crowd) · PRIVATE workforce (your employees — confidential data) · VENDOR workforces
Task types	Image classification / bounding boxes / segmentation · text classification / NER · video · custom templates
Automated data labeling	Active learning: the system trains as humans label, auto-labels the CONFIDENT items, routes only ambiguous ones to people → large-job cost drops
Ground Truth Plus	Turnkey: AWS manages the workforce and workflow end-to-end

! Confidential data → PRIVATE workforce. "Reduce labeling cost on a huge dataset" → automated data labeling (active learning). Verify predictions in PRODUCTION → that's A2I (GenAI deck), not Ground Truth.

Cleaning: duplicates, corrupt rows, outliers

◎ Detect outliers

Z-SCORE: |z| > 3 in roughly normal data.
IQR rule: outside Q1−1.5·IQR … Q3+1.5·IQR — robust, no normality assumed.
Visual: box plots.
(Streaming/anomaly at scale → Random Cut Forest, Domain 2.)

✂ Treat outliers

Remove — only if they are ERRORS (sensor glitch, impossible age 999).
Cap / winsorize — clamp to a percentile.
Transform — log shrinks right tails.
Keep — if they are the signal (fraud IS outliers!).

! Never auto-delete outliers in a FRAUD or ANOMALY problem — there, the outliers are the thing you are trying to catch.

Also under "cleaning" in the guide: DEDUPLICATION (exact + fuzzy — Glue FindMatches does ML-based fuzzy dedup) and dropping/repairing CORRUPT records (schema mismatches — caught by the validation slide ahead).

Missing data: the strategy ladder

Drop rows	Missingness is rare AND random; you can afford the loss	Cheap but biased if missingness has a pattern
Drop the column	The feature is mostly empty (e.g., >60% missing)	A hole that big rarely carries signal
Mean / median impute	Numeric; median when outliers/skew exist	Fast default; shrinks variance a bit
Mode impute	Categorical gaps	Most frequent category
Model-based (e.g., KNN)	Missingness relates to other features; accuracy matters	Best fidelity, most compute
+ Missing-indicator flag	When the FACT of being missing is informative	Add a 0/1 column alongside the imputation

! Skewed numeric with outliers → MEDIAN, not mean. And fit any imputer on the TRAINING split only — leakage rule, two slides ahead.

Scaling: standardize vs normalize — and who cares

Standardization (z-score)

x → (x − mean) / std. Centered at 0, unit variance, UNBOUNDED.

Default for gradient-descent models and anything distance-based. Robust-ish to outliers vs min-max.

Normalization (min-max)

x → (x − min) / (max − min). Squeezed into [0,1].

When a bounded range is wanted (pixels, NN inputs). FRAGILE to outliers — one extreme value crushes the rest.

WHO needs scaling:

Distance/geometry models (KNN, K-Means, SVM, PCA) and gradient-descent learners (linear/logistic, neural nets) — yes, always.
TREE models (decision trees, random forest, XGBoost) — indifferent: splits compare values within one feature only.

! Outliers present but scaling required → standardize (or a robust scaler) — NOT min-max. And fit the scaler on TRAIN only.

Reshaping features: log, binning, splitting

Log transform

For RIGHT-SKEWED positives (income, prices, counts).

Compresses the long tail toward symmetric — linear models and z-scores start behaving.

Trigger: "highly skewed".

Binning (discretize)

Numeric → categories. EQUAL-WIDTH bins vs QUANTILE bins (equal counts — better for skew).

Buys robustness to outliers & nonlinearity; costs granularity.

Trigger: "age groups", "bucket".

Feature splitting

One raw field → several usable ones: timestamp → hour/day-of-week/month; address → city/zip; name/title parsing.

Guide-named; often step 1 of real featurization.

! Skewed → log. Bucketed & outlier-robust → quantile binning. Composite raw field → split it. These three cover most "engineer a feature" stems.

D1 · TASK 1.2 · ENCODING

Encoding categoricals: the one-hot vs label trap

One-hot	One binary column per category	NOMINAL (no order): city, color. Safe default; explodes at high cardinality
Label / ordinal	Single integer code per category	ONLY when a true order exists: S<M<L, low<med<high. Otherwise it INVENTS order
Binary encoding	Integer code → its bits as columns	Middle ground for many categories: fewer columns than one-hot
Target encoding	Category → mean of the label	Powerful, HIGH leakage risk — compute within CV folds only
Hashing / embeddings	Fixed-size hash buckets · learned vectors	VERY high cardinality (user IDs, URLs); embeddings for deep models
Tokenization	Text → tokens/ids	The text-side "encoding" the guide lists (feeds BlazingText & friends)

! THE trap: label-encoding a NOMINAL feature (city=1,2,3) teaches models a fake order. No order → one-hot (or binary/hashing at scale).

Class imbalance: fixing a 99:1 world

Data-level fixes (resampling)

OVERSAMPLE the minority — duplicate or, better, SYNTHESIZE: SMOTE creates new minority points by interpolating neighbors.
UNDERSAMPLE the majority — throw data away (cheap, lossy).

Guide phrasing: "synthetic data generation, resampling" as CI strategies — for numeric, text, AND image data (augmentation).

Algorithm & metric fixes

CLASS WEIGHTS: penalize minority mistakes more (XGBoost scale_pos_weight; many built-ins expose weights).

And change the METRIC: accuracy lies at 99:1 — evaluate with precision/recall/F1/PR-AUC (Domain 2).

Resample the TRAINING split only — never the test set.

! SMOTE = synthesize minority samples. Class weights = make minority errors expensive. Test set stays UNTOUCHED and imbalanced — like reality.

Splitting: train / validation / test — done right

Hold-out 3-way	Train (fit) · Validation (tune/early-stop) · Test (touch ONCE at the end)
Stratified split	Preserve class ratios in every split — mandatory with imbalance
K-fold cross-validation	Small data: rotate the validation fold, average — stabler estimate
TIME-SERIES split	Train on the PAST, validate on the FUTURE. NEVER shuffle time — shuffling leaks tomorrow into training
Shuffle & augment	Shuffle i.i.d. data before splitting (guide-named); augment TRAIN only

! Forecasting stem + an option that says "randomly shuffle then split" → that option is the planted error. Time data splits by TIME.

Data leakage: the silent score-inflator

TARGET leakage

A feature CONTAINS the answer — because it is recorded after, or derived from, the outcome.

- "number_of_late_payment_calls" when predicting default
- "discharge_medication" when predicting diagnosis

Symptom: one feature with absurd importance; validation ≈ perfect.

TRAIN–TEST contamination

Information flows across the split:

- Scaler/imputer/encoder FIT on the full dataset
- Duplicates straddling train and test
- SMOTE applied before splitting
- Time shuffled (previous slide)

Rule: fit EVERY preprocessor on train only, inside the pipeline/folds.

! Exam symptom-sentence: "98% in validation, fails in production" → hunt the leak. Usually a post-outcome feature or a full-dataset fit.

Pre-training bias: Clarify, CI, DPL

CI Class Imbalance (CI)

Is one FACET (group) under-represented in the data itself?

Example: 90% of applicants in the dataset are from group A — the model will learn group A's world.

Fix: resampling, synthetic generation, collect more data.

DPL Difference in Proportions of Labels (DPL)

Do POSITIVE LABELS occur at different rates across groups?

Example: 40% approvals for group A vs 10% for group B in the historical labels — the model will reproduce that gap.

Detect now, or inherit it forever.

The tool is SageMaker Clarify:

you declare the FACET (the sensitive attribute) and Clarify computes pre-training metrics like CI and DPL on the dataset — before any model exists. (Post-training bias and SHAP explainability are the Domain-2 half of Clarify.)

! "Measure bias in the dataset BEFORE training" → Clarify pre-training metrics (CI, DPL) — both named verbatim in the exam guide.

Where bias comes from: selection vs measurement

Selection bias

WHO made it into the dataset is not who you'll predict for.

- Training only on approved loans (you never see rejected outcomes)
- Survey answered only by engaged users
- One region's sensors

Mitigate: representative sampling, widen collection, reweight.

Measurement bias

HOW values were captured distorts them.

- Two hospitals code the same condition differently
- Cheap sensor drifts high; labelers apply different standards
- Proxy feature measures the group, not the behavior

Mitigate: standardize instruments/guidelines, audit labels, fix proxies.

! Sampling problem (who's in) → SELECTION. Recording problem (how it's written down) → MEASUREMENT. The stem always tells you which door.

Privacy & compliance in the pipeline: PII, PHI, residency

Find PII sitting in S3 buckets (scale scan)	**Amazon Macie**
Detect / redact PII inside TEXT you process	**Amazon Comprehend (↔ GenAI deck)**
PHI in clinical text	**Comprehend Medical**
Detect & mask sensitive fields DURING ETL	**Glue sensitive-data detection / DataBrew masking**
Encrypt data at rest (S3, EBS, Feature Store, artifacts)	**AWS KMS keys**
Encrypt in transit	**TLS everywhere (default on AWS APIs)**
Keep data in a Region (data residency)	**Architecture choice: pin Region; SCP-enforce**

Anonymize BEFORE training: masking/tokenizing identifiers in the pipeline — not after the model has memorized them.

Validating quality: Glue Data Quality & DataBrew

✓ AWS Glue Data Quality

RULE-BASED validation on Catalog tables or inside Glue jobs (DQDL rules): completeness ≥ 99% · values in set · uniqueness · freshness · column count.

Can RECOMMEND rules from the data, score each run, and fail the pipeline on violation.

◔ DataBrew profiling (+ DW report)

PROFILE jobs: distributions, missing %, type inference, anomalies — the human-readable health check.

(Inside SageMaker, Data Wrangler's Insights report plays the same role for ML sets.)

! "Automatically BLOCK bad data from reaching training" → Glue Data Quality rules gating the pipeline. "Understand/profile the data" → DataBrew profile.

This is Domain 4's drift story, shifted left: the SAME idea (baseline + rules + alert) applied to incoming data instead of a live model.

D1 · SUMMARY

Domain 1 cheat map — cover the right column and self-test

Default data home · lake	**Amazon S3 (lifecycle → IA/Glacier)**
Shared POSIX files across instances	**Amazon EFS**
GPU-speed file throughput (link to S3)	**FSx for Lustre**
Big data start delay → stream instead of copy	**Pipe / FastFile input mode**
Cheap+fast analytics format on S3	**Parquet (partitioned)**
Streaming with schema evolution	**Avro**
Custom real-time consumers / replay	**Kinesis Data Streams**
"Just land the stream in S3 (as Parquet)"	**Amazon Data Firehose**
Windows / aggregations across events	**Managed Flink**
Serverless Spark ETL, least ops	**AWS Glue**
No-code visual prep (analyst)	**Glue DataBrew**
Visual prep → pipeline/Feature Store (DS in Studio)	**SageMaker Data Wrangler**
SQL on S3, serverless	**Amazon Athena**
Column-level lake permissions	**Lake Formation**
Consistent features, train & real-time	**Feature Store (offline + online)**
Label at scale, cut human cost	**Ground Truth (automated labeling)**
Skewed numeric	**Log transform · median impute**
Nominal categories	**One-hot (never label-encode)**
99:1 classes	**SMOTE / class weights (train only)**

2

ML Model Development

26% of the exam — choose, train, tune, evaluate

Frame the problem before touching a service

Predict a category (churn? fraud?)	**Classification (binary / multiclass)**
Predict a number (price, demand)	**Regression**
Group similar items, no labels	**Clustering (K-Means)**
Predict future values of a series	**Forecasting (DeepAR)**
Flag the weird ones, no labels	**Anomaly detection (RCF)**
Compress many features	**Dimensionality reduction (PCA)**
Suggest items to users	**Recommendation (Personalize / FM)**

Two guide-named selection criteria beyond accuracy:

FEASIBILITY — sometimes the answer is "you don't need ML" (a lookup table or rule solves it), or the data cannot support it.
INTERPRETABILITY — if the business must EXPLAIN each decision (loan denial reasons), linear models/trees beat deep nets and FMs.

Built-in algorithms I — tabular workhorses

XGBoost

Gradient-boosted trees — the tabular default

Classification, regression, ranking. Handles mixed features; scale_pos_weight for imbalance. If in doubt on tabular data, it's XGBoost.

Linear Learner

Linear & logistic regression at scale

Fast baseline; trains multiple models in parallel; interpretable coefficients.

K-NN

Predict from nearest neighbors

Simple classification/regression; needs scaled features; index-based.

Factorization Machines

Pairwise interactions on SPARSE data

Click prediction & recommendations where user×item matrices are huge and mostly empty.

! Tabular business data + no special constraint → XGBoost. Interpretability demanded → Linear Learner. Sparse clicks/ratings → Factorization Machines.

Built-ins II — time series & anomalies

DeepAR

PROBABILISTIC forecasting trained across MANY related series at once (all products, all stores).

Outputs quantiles — "P90 demand" — not just a point.

Trigger: hundreds of related series; confidence intervals; cold-start items.

Random Cut Forest

UNSUPERVISED anomaly detection — assigns an anomaly score per record; no labels needed.

Works on batch data and on streams (usable from Kinesis analytics).

Trigger: "detect unusual…" with NO labeled examples.

IP Insights

Learns normal (entity, IP address) pairings — user ↔ IP, account ↔ IP.

Flags logins/events from IPs unusual FOR THAT entity.

Trigger: suspicious logins, account takeover, fraud by IP behavior.

Forecast with uncertainty bands across many series → DeepAR. Unlabeled "find the weird" → RCF. Entity-vs-IP weirdness → IP Insights.

Built-ins III — text (pre-LLM, still tested)

Word embeddings (Word2Vec) + FAST supervised text classification	**BlazingText**
General-purpose embeddings of PAIRS (sentence–sentence, user–item)	**Object2Vec**
Sequence → sequence: translation, summarization (encoder–decoder)	**Seq2Seq**
Topic modeling — classical statistical	**LDA**
Topic modeling — neural variant	**NTM**

How to keep them straight:

BlazingText = words→vectors and label-a-document, at speed. Object2Vec = embeddings of any PAIRED things. Seq2Seq = text-in, text-out. LDA/NTM = discover THEMES in a corpus without labels. For modern generative text tasks, the answer is Bedrock (GenAI deck) — these appear when the stem says "built-in algorithm" or describes classic NLP pipelines.

Built-ins IV — vision: pick by OUTPUT SHAPE

Image Classification

OUTPUT: one label (or labels) for the WHOLE image.

"Is this a cat photo?"
"Which product category is pictured?"

Object Detection

OUTPUT: bounding BOXES + class per object.

"WHERE are the cars in this frame, and how many?"

Location matters, pixel precision doesn't.

Semantic Segmentation

OUTPUT: a class for EVERY PIXEL (a mask).

Medical imaging boundaries, road/lane pixels, precise crop areas.

Trigger: "pixel-level", "exact region/outline".

! Whole image → Classification. Boxes & counts → Detection. Exact outline/pixels → Segmentation. The stem's REQUIRED OUTPUT decides in one line.

D2 · TASK 2.1 · CHEAT MAP

Algorithm selection — the one-line map

Tabular prediction, default choice	**XGBoost**
Fast interpretable baseline	**Linear Learner**
Sparse clicks / user×item	**Factorization Machines**
Neighbors-based simple model	**K-NN**
Many related series + quantile forecasts	**DeepAR**
Unlabeled anomaly scores (batch or stream)	**Random Cut Forest**
Suspicious entity ↔ IP pairings	**IP Insights**
Word vectors / fast text classification	**BlazingText**
Embeddings of pairs	**Object2Vec**
Translation / summarization (built-in)	**Seq2Seq**
Discover topics, no labels	**LDA / NTM**
Group customers, no labels	**K-Means**
Reduce correlated features	**PCA**
Whole-image label / boxes / per-pixel mask	**ImgClass / ObjDetect / SemSeg**

The build-effort ladder — and cost-based selection

1	**AI Services / Bedrock / JumpStart**	No training. Managed APIs & foundation models — covered in the GenAI deck.
2	**SageMaker Canvas**	NO-CODE ML for analysts: point-and-click build, predict, deploy.
3	**Autopilot (AutoML)**	You give the dataset + target; it explores candidates, ranks them, and can generate the notebooks.
4	**Built-in algorithms**	You choose the algorithm & hyperparameters; AWS provides the optimized implementation.
5	**Script mode**	YOUR TensorFlow/PyTorch/sklearn code in AWS-managed framework containers.
6	**BYOC**	Your own Docker container — total freedom, total responsibility.

! Same rule as the GenAI deck: start at the top; go down only when a stated requirement forces you. Cost is a valid reason to stay UP.

Script mode, BYOC, and importing outside models

Script mode

Your training script + AWS's framework container.

estimator = PyTorch(entry_point="train.py", ...)
Supported: TensorFlow, PyTorch, scikit-learn, XGBoost, HuggingFace.

You write the ML; AWS owns the container, drivers, and scaling.

BYOC (bring your own container)

Your Docker image in ECR — any language, any framework, any dependency.

Required only when frameworks/deps aren't covered by managed containers.

Maximum control, maximum maintenance.

Guide-named skill — "integrate models built OUTSIDE SageMaker":

A model trained on-prem or elsewhere? Package the artifacts as model.tar.gz in S3, pair with a prebuilt inference container (or your own), create a SageMaker Model, deploy to any endpoint type. NO retraining required — importing is a packaging exercise.

! "Minimal changes to our existing PyTorch code" → script mode. "Unsupported framework/language" → BYOC. "Model trained elsewhere" → package + deploy.

Anatomy of a SageMaker training job

Estimator	**WHAT runs: algorithm/container image, IAM role, instance type & count, hyperparameters**
Input channels	**WHERE data comes from: named channels (train / validation) → S3 URIs (or EFS/FSx), input mode (File/Pipe/FastFile)**
Output	**model.tar.gz artifacts → your S3 output path (encrypt with KMS)**
Metrics	**Regex-scraped from logs → CloudWatch (objective metric for tuning reads these)**
Logs	**stdout/stderr → CloudWatch Logs (your first debugging stop)**

Why this slide matters:

Ordering questions LOVE this flow (data in S3 → estimator configured → job runs on managed instances → artifacts to S3 → model created from artifacts → endpoint). And troubleshooting stems start from "which log/metric do you check?" — CloudWatch.

D2 · TASK 2.2 · SPEED/COST

Faster & cheaper training: the three levers

Early stopping

Stop when validation stops improving.

Saves cost AND fights overfitting — one lever, two wins.

Also inside AMT: kill hopeless trials early.

Distributed training

DATA parallel: same model copied; each GPU gets different batches. Default for speed.

MODEL parallel: the model itself is split — ONLY when it cannot fit in one GPU's memory.

SageMaker has managed libraries for both.

Managed Spot

Spare capacity at up to ~90% off.

MUST pair with CHECKPOINTS to S3 — jobs can be interrupted and resume.

Set max_wait ≥ max_run. Perfect for long, interruption-tolerant training.

! "Model too large for a single GPU's memory" → MODEL parallelism — that phrase is the entire question. Spot without checkpoints = the planted flaw.

One-liner to recognize: WARM POOLS keep provisioned infrastructure alive between successive jobs — faster iteration, billed while retained.

Learning rate, batch size, epochs — cause and effect

Learning rate TOO HIGH	**Loss oscillates or DIVERGES (explodes) — overshooting minima**
Learning rate TOO LOW	**Painfully slow convergence; may stall in a poor spot**
Batch size LARGER	**Faster throughput, more GPU memory, smoother gradients — can generalize worse**
Batch size SMALLER	**Noisy updates (a mild regularizer), less memory, slower per epoch**
More EPOCHS	**Better fit until the validation curve turns — then OVERFITTING (→ early stopping)**
Steps vs epochs	**Epoch = full pass over data; step = one batch update**

Guide adds "factors that affect model size":

parameter count (layers × width), and the numeric PRECISION of weights (float32 vs float16/int8) — halve the precision, roughly halve the size. This resurfaces in the reduce-model-size slide and in Neo (Domain 3).

Automatic Model Tuning: pick the search strategy

Grid search	Try EVERY combination	**Tiny, mostly-categorical spaces only — cost explodes combinatorially**
Random search	Sample combos at random	**Cheap, fully PARALLEL, surprisingly strong baseline**
Bayesian optimization	Each trial LEARNS from previous results	**Fewest jobs to a good answer when training is expensive; inherently sequential-ish**
Hyperband	Start many, STOP losers early at checkpoints	**Best for iterative/deep models where partial results predict final quality**

Two multipliers: EARLY STOPPING inside tuning kills doomed trials; WARM START seeds a new tuning job with a previous job's results — "we tuned last month, now the data grew" → warm start, don't restart from zero.

Which knob does what — trees, networks, regularizers

Trees: MORE depth / MORE estimators	**More capacity → overfit risk rises (depth is the sharper knife)**
Trees: min samples per leaf ↑	**Smoother, simpler trees → fights overfitting**
XGBoost eta (learning rate) ↓ + more rounds	**Slower, steadier boosting — usually generalizes better**
NN: MORE layers / units	**More capacity → same overfit story; needs more data**
Dropout rate ↑	**Stronger regularization (NN-specific)**
Regularization strength (λ) ↑	**Simpler model; too far → UNDERFIT**

! Every knob is one axis: CAPACITY vs SIMPLICITY. Overfit → turn capacity down / regularization up. Underfit → the reverse.

Overfitting vs underfitting: diagnose from the curves

OVERFITTING (high variance)

Train score EXCELLENT, validation poor — a widening GAP.

Fixes: more data · augmentation · regularization (L1/L2/dropout) · early stopping · simpler model · (for FMs: see catastrophic forgetting, GenAI deck).

UNDERFITTING (high bias)

BOTH train and validation poor — the model can't even fit what it sees.

Fixes: bigger/deeper model · better features · train longer · REDUCE regularization · relevant algorithm.

The exam's twist:

"Great in validation, bad in PRODUCTION" is NOT overfitting — validation would have caught that. It's leakage (D1) or drift (D4).
Read which two environments disagree before prescribing.

! Gap between train & validation → variance → regularize/simplify/more data. Both bad → bias → add capacity. No gap analysis, no answer.

Regularization: L1 vs L2 vs dropout

L1 L1 (Lasso)

Penalizes |weights|.

Drives some weights to EXACTLY ZERO → built-in FEATURE SELECTION and sparse models.

Trigger: "many irrelevant features", "sparse/simpler model".

L2 L2 (Ridge / weight decay)

Penalizes weights².

Shrinks ALL weights small — none to zero. The general-purpose stabilizer; the guide's phrase "weight decay" = L2.

Trigger: default overfit fix.

⊘ Dropout

Randomly disables neurons each training step — the network can't rely on any single path.

NEURAL NETS ONLY (not trees).

Trigger: deep model overfitting.

! THE discriminator: need automatic feature elimination / sparsity → L1. General shrinkage → L2. Deep net → dropout. "Weight decay" on the exam = L2.

Ensembles: bagging, boosting, stacking

Bagging

Train models in PARALLEL on bootstrap samples; average/vote.

Reduces VARIANCE — tames overfitting.

Poster child: Random Forest.

Boosting

Train models in SEQUENCE; each new one focuses on the previous ones' ERRORS.

Reduces BIAS — builds a strong learner from weak ones.

Poster child: XGBoost.

Stacking

Train DIVERSE base models, then a META-MODEL learns to combine their predictions.

Squeezes the last drops of accuracy; adds complexity.

Trigger: "combine several different models".

! Parallel + variance ↓ = bagging. Sequential-on-errors + bias ↓ = boosting. Meta-model over diverse models = stacking. Three definitions, free marks.

Shrinking a model: pruning, precision, selection

Pruning	**Remove near-zero weights / whole neurons — smaller net, similar accuracy**
Lower-precision data types	**float32 → float16/int8 (quantization) — size & latency drop; slight accuracy risk**
Feature selection	**Fewer inputs (L1, importance ranking) → smaller, faster, often MORE robust**
Compression / distillation	**Train a small student to mimic the big model (distillation ↔ GenAI deck)**

Why shrink at all:

edge devices, latency SLAs, and inference cost. Domain 3's SageMaker Neo automates target-specific optimization.
Scope note: the guide lists these CONCEPTS but declares deep quantization ANALYSIS out of scope — recognize, don't compute.

! "Deploy to a device with limited memory, minimal accuracy loss" → pruning + lower-precision types (then Neo compiles it — D3).

D2 · TASK 2.2 · GOVERNANCE

Experiments & Model Registry: versioning that gates deployment

SageMaker Experiments

Auto-track every run: parameters, datasets, metrics, artifacts.

Compare runs side by side; reproduce any result.

Trigger: "track and compare training runs", "reproducibility".

Model Registry

Model package GROUPS → immutable VERSIONS with metadata, metrics, lineage.

Each version carries an APPROVAL STATUS: PendingManualApproval → Approved / Rejected.

Approval flips → EventBridge fires → CI/CD deploys.

! "Only approved models may reach production, with an audit trail" → Model Registry approval workflow. This sentence recurs across D2, D3, and D4.

Registry is the HINGE of MLOps: training pipelines END by registering a version; deployment pipelines START from an approved one. Keep this picture — Domain 3 builds the two pipelines around it.

SageMaker Debugger & Profiler: X-ray the training job

vanishing_gradient / exploding_tensor	**Gradients dying or blowing up — deep-net convergence failures**
loss_not_decreasing	**The "convergence issues" rule by name — LR wrong, data broken**
overfit / overtraining	**Validation diverging from train — stop or regularize**
class_imbalance	**Skewed labels sabotaging training (↔ D1 fixes)**
Profiler: GPU/CPU utilization	**GPU idle at 30%? The DATA PIPELINE is the bottleneck, not compute**

Debugger captures tensors DURING training, evaluates built-in RULES against them, and can auto-STOP a doomed job — saving the remaining hours of GPU money.
Rules fire → CloudWatch/EventBridge → alert or action.

Confusion matrix: precision, recall, F1 — and the threshold

Actual → Predicted ↓	+	–
+	TP	FP
–	FN	TN

The four numbers that matter

PRECISION = TP / (TP+FP) — of my alarms, how many were real?
RECALL = TP / (TP+FN) — of the real cases, how many did I catch?
F1 = harmonic mean — one number balancing both.
ACCURACY = (TP+TN)/all — MEANINGLESS under heavy imbalance.

The THRESHOLD is a business dial:

raise it → fewer alarms → precision ↑, recall ↓. Lower it → catch more → recall ↑, precision ↓.
Many "improve the model" stems are solved by MOVING THE THRESHOLD — no retraining at all.

! Guide word "heat maps" = the confusion matrix rendered as colors. In multiclass, the bright off-diagonal cell names the confused pair.

ROC-AUC vs PR-AUC — and when ROC lies

ROC curve · AUC

TPR vs FPR across ALL thresholds. AUC = probability a random positive ranks above a random negative.

0.5 = coin flip · 1.0 = perfect.

Great for BALANCED classes; threshold-independent model comparison.

Precision–Recall AUC

Precision vs recall across thresholds.

Under HEAVY IMBALANCE, ROC looks deceptively rosy (TN floods the FPR denominator) — PR-AUC stays honest because it never touches TN.

Fraud, rare disease, defects → PR-AUC.

LOG LOSS (cross-entropy):

scores the PROBABILITIES, punishing confident wrong predictions brutally. Choose it when calibrated probabilities matter — "the risk score itself feeds pricing" — not just the final label.

! Imbalanced positives + "which metric to report" → PR-AUC (or F1). ROC-AUC as the offered answer there is the trap.

Regression metrics & performance baselines

RMSE	**Same units as target; SQUARES errors → big misses dominate. Default when large errors are extra-bad**
MAE	**Mean absolute error — every unit of error equal; ROBUST to outliers**
MAPE	**Percentage error — comparable across scales; breaks near zero values**
R^2	**Fraction of variance explained vs predicting the mean; 1 = perfect, 0 = no better than mean**

BASELINES — the guide names them as a skill:

Before celebrating any model, beat the dumb answer: MAJORITY-CLASS classifier (predict the common class), MEAN predictor,
NAIVE forecast (tomorrow = today). "Is 82% accuracy good?" is unanswerable without the baseline — if majority class scores 80%, the model adds almost nothing.

D2 · TASK 2.3 · THE DECIDING DRILL

Pick the metric from the BUSINESS cost

Fraud detection	A missed fraud (FN) costs thousands; a false alarm is a review ticket	**RECALL (accept lower precision)**
Cancer screening	A missed case (FN) can be fatal; false positives get follow-up tests	**RECALL first**
Spam filter	A real email in spam (FP) loses business; spam in inbox is an annoyance	**PRECISION**
Content takedowns	Wrongly removing legit content (FP) angers users/law	**PRECISION**
Balanced stakes	Both errors comparably costly	**F1 (or cost-weighted)**

! Method: name the EXPENSIVE error first. Expensive misses → maximize recall. Expensive false alarms → maximize precision. Then tune the THRESHOLD that way.

Clarify after training: bias on predictions + SHAP

Post-training bias metrics

Now measured on the MODEL'S PREDICTIONS across facets:

DPPL — difference in positive prediction rates between groups.
DI — disparate impact ratio of those rates.

"Does the trained model approve group A more than group B?"

SHAP explanations

Feature attributions — how much each feature pushed each prediction.

GLOBAL: which features drive the model overall.
LOCAL: "THIS loan was denied because income=X, history=Y."

The answer to every "explain individual predictions" stem.

! One tool, three exam appearances: pre-training bias on DATA (D1) → post-training bias + SHAP on PREDICTIONS (here) → bias & attribution DRIFT monitoring (D4).

Domain 2 cheat map — cover and self-test

Tabular default / sparse clicks / interpretable baseline	**XGBoost / Factorization Machines / Linear Learner**
Quantile forecasts, many series	**DeepAR**
No-label anomalies / entity ↔ IP	**RCF / IP Insights**
Fast text classification / topics	**BlazingText / LDA·NTM**
Whole image / boxes / pixels	**ImgClass / ObjDetect / SemSeg**
No-code analysts / AutoML	**Canvas / Autopilot**
Your framework code / any container	**Script mode / BYOC**
Model trained elsewhere	**model.tar.gz + inference container**
~90% training discount	**Managed Spot + CHECKPOINTS**
Model exceeds one GPU's memory	**MODEL parallelism**
Expensive runs, fewest trials	**Bayesian AMT (Hyperband for iterative)**
Previous tuning job exists	**Warm start**
Gap train ↔ val / both bad	**Overfit → regularize · Underfit → capacity**
Sparsity & feature selection / weight decay / deep nets	**L1 / L2 / dropout**
Variance ↓ parallel / bias ↓ sequential / meta-combine	**Bagging / Boosting / Stacking**
Only approved models deploy	**Model Registry approval → EventBridge**
Loss flat, stop wasted GPU / GPU idle 30%	**Debugger rules / Profiler (I/O bottleneck)**
Imbalanced evaluation	**PR-AUC + F1 (never accuracy)**
Costly misses / costly false alarms	**Recall / Precision — via the THRESHOLD**

3

Deployment & Orchestration

22% of the exam — endpoints, containers, IaC, pipelines, CI/CD

Where inference runs: the four targets

SageMaker endpoints

The ML-native default

Variants, autoscaling, Model Monitor, data capture — the whole ML toolset. Pick unless a constraint pulls elsewhere.

AWS Lambda

Small CPU models, spiky traffic

Package model in the function/container image (up to 10 GB, 15-min limit). Millisecond billing, scale to zero.

ECS / EKS

You already run containers

The stem says "existing Kubernetes/ECS platform" or "standardize on containers org-wide". You own routing/scaling.

AWS Batch

Scheduled offline scoring

Containerized jobs on managed queues — nightly scoring without any endpoint.

! Default = SageMaker. Lambda for tiny/spiky CPU models. ECS/EKS only when the stem SAYS the platform already exists. Batch = offline, no endpoint.

Endpoint types — the engineer's cut

Real-time

25 MB · 60 s

Always-on fleet, ms latency. Supports variants, DATA CAPTURE (Model Monitor), autoscaling. Cannot scale to zero.

Serverless

4 MB · 60 s

Scales to ZERO; pay per use; memory 1-6 GB. COLD STARTS. No data capture — Model Monitor not supported here.

Asynchronous

1 GB · up to 1 h

Queue in front; SNS success/error notifications; can autoscale to ZERO. Big payloads, long jobs, sporadic arrivals.

Batch Transform

S3 → S3

No endpoint at all. Whole dataset scored offline; instances live only for the job.

! New vs the GenAI deck: async = queue + SNS + scale-to-zero; serverless has NO data capture (so no Model Monitor); real-time is the Monitor/variants home.

Production variants vs shadow variants

Production variants — A/B

Several model versions behind ONE endpoint, each with a traffic WEIGHT (e.g., 90/10).

Users RECEIVE responses from whichever variant served them. Per-variant CloudWatch metrics compare live performance.

Risk: the 10% experience the challenger for real.

Shadow variants — risk-free

A COPY of live traffic is mirrored to the challenger; its responses are LOGGED, never returned to users.

Compare the shadow's outputs, latency, and errors against production on identical real traffic.

Zero user impact — the guide's Task 2.3 comparison, made concrete.

! "Evaluate a new model on REAL traffic WITHOUT affecting users" → SHADOW. "Route 10% of users to the new model" → variant WEIGHTS (A/B).

D3 · TASK 3.1 · CONSOLIDATION

Many models, one endpoint: MME vs multi-container vs pipelines

Multi-model endpoint

HUNDREDS of models share one container and fleet; artifacts live in S3 and LAZY-LOAD on first call.

Same framework required. First hit to a cold model is slower.

Trigger: "a model per customer/city — thousands, most rarely called" → massive cost cut.

Multi-container endpoint

Up to 15 DIFFERENT containers (different frameworks) on one endpoint.

Invoke a specific one directly (TargetContainerHostname).

Trigger: "a PyTorch and an XGBoost model share infrastructure, called independently".

Inference pipeline

2-15 containers CHAINED in sequence on one endpoint: preprocess → predict → postprocess.

One invocation flows through all.

Trigger: "apply the SAME transforms used in training, then the model" (e.g., SparkML + XGBoost).

! Many models SAME framework, called by name → MME. FEW models DIFFERENT frameworks, independent → multi-container. Steps in SEQUENCE → inference pipeline.

Choosing inference compute — and letting AWS choose

CPU c-family (compute) / m (general)	**Classic ML (XGBoost, linear), light models — cheapest sane default**
GPU g-family	**Deep models needing acceleration at inference — cost-effective GPU**
AWS Inferentia (inf1/inf2)	**Purpose-built inference chips — best throughput per $ for supported NN models**
Graviton (ARM, e.g., c7g)	**Cheaper CPU serving for compatible stacks**
Networking note	**Distributed training wants high-bandwidth instances (EFA) — bandwidth is a guide-named factor**

Don't guess — load test:

SAGEMAKER INFERENCE RECOMMENDER runs your model against candidate instance types with real or synthetic traffic and recommends the best latency/throughput/COST configuration. "Which instance type for our endpoint?" → Inference Recommender, not intuition.

Endpoint auto scaling: metrics and policies

◎ Target tracking — the default

Keep a metric at a target value; AWS adds/removes instances.

Predefined metric: SageMakerVariantInvocationsPerInstance (e.g., hold ~1000 invocations per instance).

Custom CloudWatch metrics work too: ModelLatency, CPUUtilization, GPU util.

⇶ Step & scheduled

STEP scaling: explicit thresholds → explicit capacity jumps (alarm-driven).

SCHEDULED scaling: known patterns — scale out before the 9am rush, in at midnight.

COOLDOWNS damp flapping; min/max capacity bound the fleet.

! Real-time endpoints scale 1→N, never to zero. Async endpoints CAN reach zero (backlog-based). "Traffic doubles at predictable hours" → scheduled; otherwise target tracking.

SageMaker Neo: compile once, run optimized anywhere

Neo COMPILES a trained model (TensorFlow, PyTorch, XGBoost…) for a specific TARGET — cloud instance families or edge processors (ARM, NVIDIA Jetson, Intel) — producing a smaller, faster artifact on a lightweight runtime. Accuracy unchanged: nothing is retrained.

When Neo is the answer

"Run the model ON the device / in the factory with limited compute" · "reduce inference latency and model footprint for target hardware" · pairs with D2's pruning and data-type shrinking.

Scope alert

AWS IoT Greengrass — the edge DEPLOYMENT runtime — is OUT OF SCOPE for MLA-C01. If it appears in options, it is a distractor. Neo (the optimizer) is the in-scope edge answer.

! Edge stem → optimize with Neo. Do not reach for Greengrass on THIS exam — the guide excludes it explicitly.

Containers for serving: DLC → extend → BYOC

Prebuilt DLCs	AWS framework images (TF/PyTorch/sklearn/XGBoost…) — zero image work; pass your model and code
Extend a DLC	FROM the AWS image, pip install extras — when only a few packages are missing
BYOC	Your Dockerfile in ECR. Serving contract: web server on port 8080 answering POST /invocations and GET /ping

The ECR flow:

build image → push to Amazon ECR → reference the image URI in the SageMaker Model/estimator. CodeBuild automates build+push inside CI/CD (two slides ahead). Training containers mirror the contract: read /opt/ml/input, write the model to /opt/ml/model.

! Ladder rule again: DLC if possible, extend if a few deps are missing, BYOC only when the stack is unsupported. The /invocations + /ping pair is a free recognition point.

The deploy triad — and zero-downtime updates

1 · Model	artifacts (model.tar.gz in S3) + container image + execution role
2 · EndpointConfig	which model(s), variants and weights, instance type/count (or serverless/async settings)
3 · Endpoint	the live HTTPS resource created FROM a config

Updating without downtime:

create a NEW EndpointConfig (new model version) and call UpdateEndpoint. SageMaker stands up the new fleet, shifts traffic, then retires the old — a managed blue/green swap. Rollback = update back to the previous config. NEVER delete and recreate the endpoint.

Deployment strategies: blue/green, canary, linear, rolling

Blue/green ALL-AT-ONCE	New fleet up → 100% traffic flips → old drains. Fastest, biggest blast radius
Blue/green CANARY	Small slice (e.g., 10%) first → BAKE period under alarms → then the rest
Blue/green LINEAR	Traffic moves in equal steps (20%… 40%…) with a bake between each
Rolling	Replace instances in batches within the fleet — less extra capacity than blue/green
Auto-ROLLBACK	CloudWatch ALARMS watch during shifts; any breach → automatic revert to the old version

! "Minimize risk of a bad model reaching all users, with AUTOMATIC rollback" → blue/green canary + CloudWatch alarms. Version everything so rollback is a pointer flip.

Endpoints inside your network: the VPC config

VpcConfig on models/jobs

Attach SUBNETS + SECURITY GROUPS to hosting and training.

The containers' network interfaces (ENIs) live in YOUR VPC — traffic to your databases/APIs stays private, governed by your SGs.

What the model can reach

With VPC placement, reach S3 via a GATEWAY VPC endpoint and other AWS APIs via INTERFACE endpoints — no internet route needed.

(Full hardening — isolation, PrivateLink details — is Domain 4.)

! "The endpoint must access an RDS database in our private subnets" / "no internet-facing paths" → VpcConfig with subnets + security groups.

Infrastructure as code: CloudFormation vs CDK

AWS CloudFormation

DECLARATIVE templates (YAML/JSON) → stacks.

Repeatable environments, change sets preview, drift detection, StackSets across accounts.

Cross-stack wiring: Outputs/Exports + Fn::ImportValue.

AWS CDK

Define infrastructure in REAL CODE (Python/TypeScript) — loops, conditionals, reusable constructs — which SYNTHESIZES to CloudFormation.

Trigger: "developers prefer familiar programming languages", complex parameterized stacks.

! CDK compiles TO CloudFormation — one deployment engine, two authoring styles. "Declarative templates" → CFN. "Use Python to define infra" → CDK.

Pipelines vs Step Functions vs MWAA vs EventBridge

SageMaker Pipelines	ML-NATIVE DAGs: processing→training→evaluate→register. Step CACHING, Studio graph, Registry integration. Default for pure-ML workflows
Step Functions	Orchestrate ANY AWS services with branching, retries, error handling, human callbacks — ML steps mixed with Lambda/Glue/ECS
Amazon MWAA (Airflow)	The team ALREADY has Airflow DAGs / Python-operator ecosystem. Managed Airflow; always-on environment cost
EventBridge	NOT an orchestrator — the TRIGGER: schedules (cron) and event rules that START the others

! Pure ML steps → Pipelines. Mixed AWS services / complex error handling → Step Functions. "Existing Airflow" → MWAA. "Every night at 2am" / "when X happens" → EventBridge starts it.

Inside SageMaker Pipelines

ProcessingStep	**Data prep / feature engineering / EVALUATION scripts (or a Data Wrangler export)**
TrainingStep / TuningStep	**A training job or a full AMT sweep as one node**
ConditionStep	**The QUALITY GATE: proceed only if metric ≥ threshold**
RegisterModel	**Push the candidate into the Registry (PendingManualApproval)**
Parameters & caching	**Parameterize instance types/paths; unchanged steps are SKIPPED on re-run**

The canonical exam DAG:

process data → train → evaluate (Processing) → Condition: accuracy above bar? → RegisterModel → (human approves)
→ deployment pipeline takes over. Ordering questions reuse this sequence almost verbatim.

CI/CD for ML: CodePipeline, CodeBuild, CodeDeploy

CodePipeline

The ORCHESTRATOR of release stages: Source → Build → (Approve) → Deploy.

Manual APPROVAL actions gate production.

Starts on Git pushes or EventBridge events.

CodeBuild

Managed build/test runners (buildspec.yml).

In ML: run unit tests, BUILD & PUSH containers to ECR, kick off SageMaker Pipelines, run integration tests.

CodeDeploy

Deployment automation for EC2/ECS/Lambda with canary/linear shifting.

SageMaker endpoints usually deploy via pipeline steps / CFN + guardrails instead.

Git flows (guide-named): GITFLOW — long-lived develop/release branches, scheduled releases. GITHUB FLOW — short feature branches → PR → merge to main → deploy; the continuous-delivery default. Each Code* service has QUOTAS (pipelines/stages/actions) — know they exist.

! "On every merge: test → build image → retrain → require human sign-off → deploy" = CodePipeline stages wrapping CodeBuild + a SageMaker Pipeline + an Approval action.

The full MLOps loop — with tests where they belong

commit → CodeBuild: UNIT tests + build containers → SageMaker Pipeline: process → train → evaluate → Condition gate → RegisterModel → human APPROVAL → deploy (blue/green canary) → E2E smoke test → Model Monitor watches (D4)

UNIT tests	**Feature-engineering functions, schema validators — fast, in CodeBuild, every commit**
INTEGRATION tests	**Steps together: container starts, pipeline runs on a SAMPLE, invoke a staging endpoint**
END-TO-END tests	**Full path on real-shaped data: deploy to staging → send requests → assert outputs and latency**

SageMaker PROJECTS: MLOps starter templates — repos + this whole pipeline pre-wired via CloudFormation. "Stand up standardized MLOps quickly" → Projects.

Automated retraining: the three triggers

Schedule

EventBridge cron rule → start the training pipeline.

"Retrain weekly on the latest data."

Simple, predictable, may retrain needlessly.

Drift-driven

Model Monitor violation → CloudWatch/EventBridge → pipeline.

"Retrain WHEN performance degrades."

Retrains exactly when needed — the D4 handshake.

Data/event-driven

New data lands (S3 events — including via CloudTrail data events) → EventBridge rule → pipeline.

The guide's "use CloudTrail to invoke re-training" is this exact pattern.

! All three share one skeleton: SIGNAL → EventBridge rule → SageMaker Pipeline execution → gate → registry → deploy. Only the signal changes.

Domain 3 cheat map — cover and self-test

Default ML hosting (variants, Monitor, autoscale)	**SageMaker endpoints**
Tiny CPU model, spiky, scale-to-zero	**Lambda**
"Existing Kubernetes/containers platform"	**ECS / EKS**
Test on real traffic, ZERO user impact	**Shadow variant**
Route 10% of users to the challenger	**Production variant weights (A/B)**
Thousands of same-framework models, most idle	**Multi-model endpoint**
Preprocess → model → postprocess chained	**Inference pipeline**
Best throughput/$ for NN inference	**Inferentia (inf)**
"Which instance type?" — don't guess	**Inference Recommender**
Default scaling metric	**InvocationsPerInstance (target tracking)**
Optimize for edge (Greengrass = out of scope)	**SageMaker Neo**
BYOC serving contract	**/invocations + /ping on :8080**
Zero-downtime model swap	**New EndpointConfig + UpdateEndpoint**
Safe rollout + auto-rollback on alarms	**Blue/green canary/linear (guardrails)**
Declarative templates / infra in Python	**CloudFormation / CDK**
ML-native DAG, caching, quality gate	**SageMaker Pipelines (ConditionStep)**
Mixed-service workflow, retries, callbacks	**Step Functions**
"Existing Airflow"	**MWAA**
Cron / event that STARTS pipelines	**EventBridge (trigger, not orchestrator)**

4

Monitoring, Maintenance & Security

24% of the exam — drift, costs, IAM, VPC, encryption

Why models rot: the five drifts

Data (covariate) drift	The INPUTS' distribution shifts — new demographics, new price ranges, a schema change
Concept drift	The X→y RELATIONSHIP changes — fraud tactics evolve; the same inputs now mean something else
Model quality drift	The ACCURACY/F1 itself degrades — measurable only once ground-truth labels arrive
Bias drift	Fairness metrics shift across groups in live traffic
Feature attribution drift	SHAP importance rankings change — the model is 'reasoning' differently than at training

! Inputs changed → data drift. Meaning changed → concept drift. Scores dropped (needs labels!) → quality drift. Each maps to ONE Model Monitor type — next slides.

Model Monitor: capture → baseline → schedule → violations

1 · Data Capture	Enable on the endpoint: a % of requests+responses is copied to S3 (real-time & batch transform; NOT serverless)
2 · Baseline job	Run against the TRAINING data → statistics.json + constraints.json (the definition of 'normal')
3 · Monitoring schedule	Hourly/daily jobs compare captured traffic vs the baseline
4 · Violations	A violations report + CloudWatch metrics → alarm / EventBridge → notify or RETRAIN

! Baseline from TRAINING data = the yardstick. No data capture → no Model Monitor (that's why serverless endpoints are excluded).

Recognize this as the D1 quality-gate pattern shifted right: baseline + rules + alert — applied to LIVE traffic instead of incoming files.

The four monitor types — and the ground-truth catch

Data Quality	Live inputs vs training statistics: types, ranges, missing %, distribution distance	**Catches DATA drift early — no labels needed**
Model Quality	Live accuracy/F1/RMSE — requires you to MERGE ground-truth labels with captured predictions	**The catch: labels arrive LATE (label lag)**
Bias Drift	Clarify's fairness metrics recomputed on live traffic	**"Has the model become unfair in production?"**
Feature Attribution Drift	Live SHAP rankings vs the training-time explainability baseline	**"Is it reasoning differently than validated?"**

! "Track live ACCURACY" → Model Quality monitor + a ground-truth ingestion pipeline. No labels available yet? Then you can only watch DATA quality — that IS the design.

From alert to action: the automated loop, assembled

Model Monitor violation → CloudWatch metric/alarm → EventBridge rule → ① SNS notify the team
② start the SageMaker retraining Pipeline → evaluate gate → Registry approval → blue/green canary deploy

Why this exact chain

Each piece is the least-ops tool for its job: Monitor detects, CloudWatch quantifies, EventBridge routes, Pipelines rebuilds, Registry gates, guardrails ship safely.

The planted flaws

"Retrain and auto-deploy with no evaluation gate" (governance failure) · "a Lambda polls metrics on a cron" (hand-built EventBridge) · "wait for users to complain" (not monitoring).

! This one diagram answers every "automatically detect and remediate drift" stem. Memorize it as a sentence you can recite.

A/B testing in production — the monitoring angle

The mechanics (from D3)

Two production variants, weighted 90/10.

CloudWatch reports Invocations, ModelLatency, and errors PER VARIANT; your app logs business outcomes per variant.

Promotion = shift weights; failure = weight back to zero.

A/B vs shadow — final word

SHADOW answers "is it safe/technically better?" with zero user exposure.

A/B answers "does it move USER outcomes?" — click-through, conversion — which only real exposure can measure.

Mature flow: shadow first, then A/B, then full rollout.

! "Measure the new model's effect on conversion with limited exposure" → A/B (weights). "Validate before any user sees it" → shadow. Many stems test exactly this line.

CloudWatch for ML: the metrics that matter

ModelLatency vs OverheadLatency	**Time INSIDE your container vs time in SageMaker's routing — the blame-assignment pair**
Invocations / InvocationsPerInstance	**Traffic volume; the second drives autoscaling (D3)**
Invocation4XXErrors / 5XXErrors	**Client-side vs server-side failures — the triage fork**
CPU / GPU / Memory utilization	**Right-sizing evidence; GPU idle = data-pipeline bottleneck (D2 Profiler echo)**
Alarms · Dashboards	**Thresholds that page or trigger; single-pane views per endpoint**
Logs Insights · Lambda Insights	**Query endpoint/job logs at scale; deep metrics for Lambda-hosted pieces (guide-named)**

! High ModelLatency → optimize the MODEL (Neo, better instance). High OverheadLatency → look at payload size and request path. This split is a documented question pattern.

D4 · TASK 4.2 · AUDIT & TRACE

Who did what, and where did the time go: CloudTrail vs X-Ray

AWS CloudTrail — the AUDIT log

Records API CALLS: who created/deleted the endpoint, who changed the config, from where, when.

Trails persist to S3 (immutable evidence). And per the guide, its events can INVOKE RETRAINING via EventBridge (D3 trigger #3).

AWS X-Ray — the TRACE

Follows ONE REQUEST across services — API Gateway → Lambda → SageMaker endpoint — timing each hop.

"Users report slowness; WHICH component adds the latency?" → X-Ray's service map answers it.

One-liners in scope: AWS CONFIG — records resource configurations & evaluates compliance rules over time. QUICKSIGHT — dashboards for business/monitoring reporting (guide-named for visualizations).

! WHO changed it → CloudTrail. WHERE is the latency across services → X-Ray. IS the resource compliant → Config. Three verbs, three services.

The troubleshooting playbook

4XX errors	**Caller's fault: payload format, content type, auth/IAM — inspect a failing request**
5XX errors / container crash	**Read endpoint CloudWatch LOGS; OOM → bigger instance or smaller batch/model**
ThrottlingException	**Service QUOTAS hit or under-scaled → request quota increase / autoscaling**
Serverless cold starts	**First-hit latency after idle → PROVISIONED CONCURRENCY (guide-named)**
High ModelLatency	**The model itself: right-size (Inference Recommender), optimize (Neo), shrink (D2)**
High OverheadLatency	**Payload too big, request path — not the model**
GPU util low, cost high	**I/O-starved serving/training → data pipeline, not more GPU**

! Discipline: METRIC first (which latency? which error class?), LOGS second, FIX third. Options that jump to "redeploy on bigger instances" without diagnosis are the trap.

Instance families: match the letter to the workload

m — general purpose	**Balanced CPU/RAM — notebooks, light serving, the safe default**
c — compute optimized	**CPU-hungry serving (tree/linear models at volume), preprocessing**
r — memory optimized	**Big datasets in RAM, memory-heavy transforms and models**
p — GPU (training class)	**Deep-learning TRAINING horsepower**
g — GPU (inference/graphics class)	**Cost-effective GPU INFERENCE**
inf — Inferentia	**The guide's "inference optimized": purpose-built chips, best NN throughput per $**

! The guide's exact words: "memory optimized, compute optimized, general purpose, inference optimized." Map each phrase to r / c / m / inf and the family questions are free.

Inference Recommender vs Compute Optimizer

SageMaker Inference Recommender

Scope: SAGEMAKER ENDPOINTS.

Load-tests YOUR model across candidate instance types (and serverless configs) against your latency/throughput targets → ranked recommendations with projected cost.

"Which instance for this endpoint?"

AWS Compute Optimizer

Scope: THE FLEET — EC2, Auto Scaling groups, EBS, Lambda.

Analyzes historical utilization and flags over/under-provisioned resources account-wide.

"Right-size our compute estate."

! Endpoint sizing → Inference Recommender (it TESTS the model). General infrastructure right-sizing → Compute Optimizer (it READS utilization history). Both are guide-named.

Cost levers — training side

Managed Spot + checkpoints	**Up to ~90% off interruption-tolerant jobs (the D2 pattern — checkpoints are mandatory)**
Stop idle notebooks	**Lifecycle configurations auto-stop idle Studio/notebook instances — the classic silent leak**
Right-size the job	**Profiler/CloudWatch evidence → smaller or better-matched instances**
Early stopping & AMT strategy	**Fewer wasted epochs; Bayesian/Hyperband = fewer jobs (D2 echoes)**
Pipeline step caching	**Skip recomputing unchanged prep steps (D3 echo)**
Warm pools — with eyes open	**Faster iteration between jobs, but retained infra BILLS — a tradeoff, not free**

"Data scientists leave notebooks running overnight" → lifecycle configs that auto-stop on idle. This exact stem recurs.

Cost levers — inference side, and the purchase options

Serverless / async for spiky-sparse	**Scale to zero — stop paying for idle real-time fleets**
Multi-model endpoints	**Consolidate hundreds of low-traffic models onto one fleet (D3)**
Autoscaling tuned	**Sane min capacity; scale in when quiet**
Cheaper silicon	**Graviton CPU · g-family or Inferentia for NN inference**
Delete the forgotten	**Unused endpoints bill 24/7 — Trusted Advisor / Cost Explorer find them**

Purchase options — the guide names all four:

ON-DEMAND (default, flexible) · SPOT (TRAINING ONLY — never hosting) · RESERVED INSTANCES (EC2-side; do NOT cover SageMaker)
· SAGEMAKER SAVINGS PLANS — commit $/hr for 1 or 3 years, up to ~64% off, covers SageMaker usage broadly (endpoints, training, notebooks).

Cost governance: see it, allocate it, cap it

Cost allocation TAGS	Tag every job/endpoint (team, project, env) — the prerequisite for attribution
Cost Explorer	Analyze and filter spend BY TAG, service, time; find the expensive endpoint
AWS Budgets	Thresholds + ALERTS ("ML spend > $10k → notify") — proactive, not forensic
Trusted Advisor	Checks for idle/underutilized resources, plus security & limits
Billing / Cost & Usage	The raw ledger feeding all of the above

! "Which TEAM is driving the SageMaker bill?" → tags + Cost Explorer. "Alert before we overspend" → Budgets. "Find idle resources" → Trusted Advisor.

IAM for ML: two roles, least privilege, Role Manager

The two-role picture

USER/caller permissions: may THIS person call CreateTrainingJob / InvokeEndpoint?

EXECUTION ROLE: what the JOB/ENDPOINT itself may touch — read the S3 data, pull the ECR image, write logs, use the KMS key.

Most "AccessDenied" stems are the EXECUTION role missing a permission.

Least privilege, practically

Scope roles to specific buckets/prefixes and keys — no wildcards.
Groups for human permissions; resource policies (BUCKET POLICIES) on the data side.

SAGEMAKER ROLE MANAGER: generates least-privilege roles from PERSONAS (data scientist, MLOps) — the fast, correct answer for "set up roles quickly and safely".

! "Training job can't read S3 / pull the image" → fix the EXECUTION role. "Standardized least-privilege roles per team persona" → SageMaker Role Manager.

Locking the network: VPC-only, isolation, encrypted links

VPC-only workloads	**Training/hosting/Studio launched with VpcConfig — your subnets & security groups (D3 plumbing, now policy)**
No direct internet	**Private subnets, no IGW path; reach AWS services via VPC endpoints only**
Network ISOLATION	**EnableNetworkIsolation: the container gets NO outbound network at all — even you can't exfiltrate**
Inter-container encryption	**Encrypt traffic BETWEEN distributed-training instances (compliance; costs some speed)**
Security groups & subnets	**The per-workload firewall and placement — least-open rules, private tiers**

! "Training on highly sensitive data must have NO possible network egress" → VPC-only + EnableNetworkIsolation. "Encrypt traffic between training nodes" → inter-container encryption.

Private connectivity: PrivateLink & VPC endpoints

Interface endpoints (PrivateLink)

Private ENIs in your subnets for AWS APIs:

- sagemaker.api — control plane (create/manage)
- sagemaker.runtime — INVOKE ENDPOINTS privately
- Studio, and most other AWS services

Calls never touch the public internet.

Gateway endpoint + on-prem

S3 (and DynamoDB) use GATEWAY endpoints — route-table entries, no charge.

From ON-PREM: Direct Connect (or VPN) into the VPC, then the same private endpoints — a fully private hybrid path.

! "Invoke the endpoint / reach S3 without traversing the internet" → interface endpoint for sagemaker.runtime + S3 gateway endpoint. Free recognition marks.

Encryption everywhere — and the KMS gotcha

S3 datasets & artifacts	**SSE-KMS with your CMK; bucket policy can DENY unencrypted puts**
Training/processing volumes	**VolumeKmsKeyId encrypts the attached EBS storage of jobs**
Endpoints, notebooks, Feature Store	**KMS keys on their storage too — 'encrypt at rest' is a checkbox, use it**
In transit	**TLS on every AWS API and endpoint call (+ inter-container encryption from the network slide)**

THE GOTCHA the exam loves:

Encrypting with a customer-managed key means every EXECUTION ROLE that touches the data needs kms:Decrypt / GenerateDataKey on THAT KEY (via key policy or IAM). "Job fails with AccessDenied although the S3 policy is correct" → the KMS key permissions.

Secrets & CI/CD hygiene

Secrets Manager vs Parameter Store

SECRETS MANAGER: credentials with automatic ROTATION (database passwords, API keys) — the word "rotate" selects it.

SSM PARAMETER STORE: configuration values; SecureString for encrypted params; simpler and cheaper when rotation isn't needed.

Pipeline security rules

NEVER hardcode credentials in code, notebooks, or images — fetch at runtime from Secrets Manager/SSM via the role.

Least-privilege PIPELINE roles · locked-down artifact buckets · manual APPROVAL gates before production (guide-named CI/CD best practices).

! "Training code needs the database password, rotated automatically, never in the repo" → Secrets Manager, fetched at runtime by the execution role.

Audit & compliance: prove what happened

CloudTrail TRAIL → S3	**Durable, validated API history — who did what, forever (single-account or org-wide)**
CloudWatch Logs retention	**Set retention policies on job/endpoint logs to match compliance periods**
AWS Config rules	**Continuous compliance: "volumes encrypted", "no direct internet on notebooks" — flag or auto-remediate**
Organizations SCPs	**Guardrails above IAM: deny regions (residency), deny disabling CloudTrail**
Registry + lineage	**Which model version served, built from which data/code — the ML-specific audit answer**

! "Prove to auditors which model version made the decision and who approved it" → Model Registry (+ lineage) with CloudTrail — not CloudWatch metrics.

Domain 4 cheat map — cover and self-test

Inputs shifted / meaning shifted / accuracy shifted	**Data drift / concept drift / model-quality drift**
Monitor prerequisite on the endpoint	**Data Capture (not on serverless!)**
"Normal" is defined by…	**Baseline from TRAINING data (statistics + constraints)**
Live accuracy monitoring needs…	**Ground-truth labels merged (label lag!)**
Drift → automatic retrain	**Violation → EventBridge → Pipeline → gate → deploy**
Container time vs routing time	**ModelLatency vs OverheadLatency**
4XX / 5XX / Throttling	**Caller / container-logs+OOM / quotas+autoscaling**
Serverless cold starts	**Provisioned concurrency**
Who changed it / where's the latency / is it compliant	**CloudTrail / X-Ray / Config**
"Which instance for the endpoint?"	**Inference Recommender (fleet-wide → Compute Optimizer)**
Idle notebooks overnight	**Lifecycle configs auto-stop**
Commitment discount on SageMaker	**SageMaker Savings Plans (NOT EC2 RIs; Spot = training only)**
"Which team spends what" / "alert on overspend"	**Tags + Cost Explorer / Budgets**
Job can't read S3/ECR	**Fix the EXECUTION role**
Personas → least-privilege roles fast	**SageMaker Role Manager**
Zero possible egress from training	**VPC-only + network isolation**
Invoke privately, no internet	**PrivateLink runtime endpoint + S3 gateway**
Encrypted data still AccessDenied	**KMS key permissions for the role**
"Rotate the credentials"	**Secrets Manager (config → Parameter Store)**

SUMMARY · CROSS-DOMAIN

Fifteen rules that carry the ML 90%

1. Constraint is in the last sentences; extract 2-3, eliminate one option per constraint.
2. Managed beats hand-built: Feature Store, Model Monitor, AMT, Role Manager over DIY glue.
3. Out-of-scope services in options are distractors: Greengrass, GuardDuty, Inspector, Cognito, WAF.
4. Analytics on S3 → Parquet + partitions. Big-data training start → Pipe/FastFile, not File.
5. Streams = your consumers/replay; Firehose = deliver-to-destination with zero code.
6. Never shuffle time series; fit every preprocessor on TRAIN only; resample train only.
7. Perfect validation + failing production = leakage (D1) or drift (D4) — not overfitting.
8. Tabular default XGBoost; no labels → clustering/RCF; many related series → DeepAR.
9. Spot training needs checkpoints; model-too-big-for-GPU = model parallelism.
10. Imbalanced data: SMOTE/weights to train, PR-AUC/F1 to evaluate, threshold to tune.
11. Only approved Registry versions deploy — the gate survives every automation.
12. Zero user impact → shadow; user outcomes → A/B; safe rollout → blue/green canary + alarm rollback.
13. Pipelines for pure ML; Step Functions for mixed; EventBridge triggers, never orchestrates.
14. Monitor = capture → baseline → schedule → violations; live accuracy needs ground-truth labels.
15. SageMaker discounts: Savings Plans (hosting+training), Spot (training only), RIs (never).

SUMMARY · THE PLAN

How to study this deck

1 · Sequence	D1 → D2 → D3 → D4, one domain per sitting. Read slide THEN its Study Notes — the notes carry half the content.
2 · Self-test	After each domain: cover the cheat map's right column and answer in under two seconds, both directions.
3 · Space it	Re-read the whole deck once, 48 hours later. Spaced repetition beats a second consecutive pass.
4 · Drill	Then move to the question bank (exam-length, multi-constraint stems) and diagnose every miss back to a slide.
5 · Pair it	This deck + the GenAI deck = the full MLA-C01 surface. Cross-references between them are deliberate — follow them.

! Exam day: 130 minutes, 65 questions = 2 min each. Flag and move on; unanswered = wrong, guessed = maybe right. No penalty for guessing.

YOU NOW HOLD THE FULL MAP

Prepare. Build. Ship. Operate.

Four domains, one lifecycle, one reading method: locate the station, extract the constraints, eliminate.
Every service in this deck is a station on that line — and now you know which one, and why.

Next stop: the question bank — exam-length scenarios, all five formats. Score 80%+ twice, 48 hours apart, and book the exam.

Good luck.